



Cloud-native · Event-driven · Kubernetes-orchestrated

Production-style distributed backend built with Node.js, Docker & Minikube

*Source: commit **eddd9d5** — github.com/rajasekar21/ride-hailing-platform*

TEAM MEMBERS

- Akanksha Rathore
- Manesh Nivas Borkar
- N K Anil Kumar
- Saif Uddin Ahamed
- Rajasekar S

Team-26 · 2025

Table of Contents

1. Architecture Overview	3
2. System Components	3
3. Tech Stack	4
4. Dataset Integration	4
5. Setup Instructions	4
6. API Endpoints	5
7. Autoscaling (HPA)	5
8. Authentication & Security	5
9. Troubleshooting	6
10. Repository Structure	6
11. Future Enhancements	6
12. Conclusion	6
13. Task Distribution	7
14. Contributors	7



1. Architecture Overview

This system follows a **microservices architecture** with independently deployable, loosely coupled services — each owning its own SQLite database. Inter-service communication is event-driven via RabbitMQ, containers are managed with Docker, and orchestration is handled by Kubernetes (Minikube).

Event Flow

- User → Ride Service → Driver Service → Payment Service → Notification Service

2. System Components

Backend Microservices

- **User Service** — Manages rider profiles
- **Driver Service** — Handles driver allocation
- **Ride Service** — Full ride lifecycle management
- **Payment Service** — Payment processing
- **Notification Service** — Event-triggered notifications
- **Auth Service** — JWT-based authentication

Infrastructure

- **Message Broker** — RabbitMQ
- **Cache Layer** — Redis
- **Database** — SQLite (per service)
- **API Gateway** — Nginx (optional)

Frontend

- React Dashboard with live ride booking simulation
- Analytics: rides, revenue, system metrics
- Map-based visualization via Leaflet

3. Tech Stack

Layer	Technology
Backend	Node.js (Express)
Frontend	React
Database	SQLite (per-service)
Messaging	RabbitMQ
Cache	Redis
Container	Docker
Orchestration	Kubernetes / Minikube

4. Dataset Integration

Real CSV datasets power realistic simulation. Each service seeds its own SQLite database from the shared dataset/ directory.

Structure

```
dataset/
├── users.csv
├── drivers.csv
├── rides.csv
└── payments.csv
```

Seed command (run inside each service directory):

```
node seed.js
```

5. Setup Instructions

Prerequisites

- Node.js v18+
- Docker
- Minikube
- kubectl
- Git

Step 1 — Start Minikube

```
minikube start --memory=4096 --cpus=2
minikube addons enable metrics-server
eval $(minikube docker-env)
```

Step 2 — Build Docker Images

```
docker build -t user ./services/user
docker build -t ride ./services/ride
docker build -t driver ./services/driver
docker build -t payment ./services/payment
docker build -t notification ./services/notification
docker build -t auth ./services/auth
docker build -t frontend ./frontend
```

Step 3 — Deploy to Kubernetes

```
kubectl apply -f k8s/
```

Step 4 — Verify & Access

```
kubectl get pods && kubectl get svc
minikube ip
# Frontend → http://<IP>:30010 API → http://<IP>:30000
```

6. API Endpoints

User Service	GET	/users
User Service	POST	/users
Ride Service	POST	/rides
Ride Service	GET	/rides
Auth Service	POST	/login

All Services

GET

/metrics

7. Autoscaling (HPA)

Enable HPA for the Ride service:

```
kubectl autoscale deployment ride \
--cpu-percent=50 --min=1 --max=5
```

Generate load & monitor:

```
while true; do curl -X POST http://<IP>:30000/rides; done
kubectl get hpa # watch replica count
kubectl get pods -w
```

8. Authentication & Security

- JWT-based login via the Auth Service
- Role-based access control: **Admin**, **Driver**, **Rider**

9. Troubleshooting

Pods not starting

```
kubectl describe pod <pod-name>
```

View service logs

```
kubectl logs <pod-name>
```

Restart cluster

```
minikube delete && minikube start
```

10. Repository Structure

```
ride-hailing-platform/
├── services/ # Microservice source code
├── frontend/ # React dashboard
├── k8s/ # Kubernetes manifests
├── dataset/ # CSV seed data
├── scripts/ # run-all.sh & helpers
└── README.md
```

11. Future Enhancements

- WebSockets for real-time driver tracking
- UPI payment simulation
- Prometheus + Grafana monitoring
- Cloud deployment (AWS / GCP)
- CI/CD pipeline automation

12. Conclusion

This project demonstrates a **production-style ride-hailing platform** using modern cloud-native technologies, emphasising scalability, modularity, and real-world system design principles. The event-driven microservices architecture ensures loose coupling, independent deployability, and horizontal scalability through Kubernetes HPA.

13. Task Distribution

The following table summarises each team member's primary ownership and contribution area for this project.

#	Contributor	Tasks & Responsibilities
1	Akanksha Rathore	User Service (REST API, SQLite schema, seed data), Frontend React dashboard layout, Leaflet map integration
2	Manesh Nivas Borkar	Driver Service (allocation logic, RabbitMQ consumer), Docker image builds, Kubernetes deployment manifests (k8s/)
3	N K Anil Kumar	Ride Service (lifecycle management, ride_queue publisher), Dataset CSV generation, DB seeding scripts
4	Saif Uddin Ahamed	Payment Service (payment_queue consumer), Notification Service, RabbitMQ broker setup, Redis cache layer
5	Rajasekar S	Auth Service (JWT, role-based access), Minikube HPA configuration, CI/CD scripts, overall architecture & README

14. Contributors

1	2	3
Akanksha Rathore	Manesh Nivas Borkar	N K Anil Kumar
4	5	
Saif Uddin Ahamed	Rajasekar S	

github.com/rajasekar21/ride-hailing-platform · Commit eddd9d5